



Universidad Autónoma de Nayarit

Unidad Académica De Economía

Sistemas Computacionales

“Semana 14 de 14 • Integración de Múltiples Estructuras”

Jorge Armando Navarrete Ruiz

Docente.- DR. Eligardo Cruz Sanchez

U. A. ESTRUCTURA DE BASES DE DATOS

Fase 1: COMPRENDE	3
Fase 2: APLICA	4
Fase 3: REFLEXIONA	5
Observación P95 vs Mediana:	5
# - lookup: P95 cercano a la mediana (bajo jitter).	5
# - rango/top10: P95 mayor por variación del SO/cache y por procesar más datos (k) y ordenar.	5
#	5
# Cuello de botella (n=10,000):	5
# - rango (Mediana 808.8930 us) es el más costoso -> domina el tiempo del sistema.	5
Fase 4: VALIDA	6

Fase 1: COMPRENDE

```
# COMPRENDE-1
# Inventario de Estructuras del Proyecto (SistemaMX)
#
# | Estructura | Datos que almacena |
# |-----|-----|
# | unordered_map<string, Pelicula> | Catálogo: película por ID |
# | BST (NodoBST por puntaje) | Índice secundario: películas agrupadas por puntaje |
# | vector<vector<Pelicula>> pilaResultados | Snapshots de resultados de consultas 'rango' |
# | vector<OperacionUndo> pilaUndo | Historial de inserciones (IDs insertados) |
```

Operación principal que resuelve	Complejidad esperada	Semana origen
lookup exacto por ID	$O(1)$ amortizado	S12
búsqueda por rango [minv,maxv] de puntajes	$O(h + k)$ (promedio $\sim O(\log n + k)$)	S13
reutilizar la última consulta para 'top10'	push_back $O(1)$, acceso back $O(1)$	S5 (pila)
deshacer última inserción (undo)	push_back $O(1)$, pop_back $O(1)$	S5 (pila)

COMPRENDE-2

Matriz de acoplamiento / Operaciones compuestas (mínimo 2)

#

Operación compuesta 1: rango(minv,maxv)

Paso 1: recorrer BST y recolectar resultados en rango

bst_rango(bst_raiz, minv, maxv, resultados) -> $O(\log n + k)$ promedio (BST), peor $O(n + k)$

Paso 2: guardar snapshot para consultas posteriores

pilaResultados.push_back(resultados) -> $O(1)$
(amortizado) + copia $O(k)$

Complejidad total: $O(\log n + k)$ (dominante: recorrido + k resultados)

#

Operación compuesta 2: top10() (depende del último rango)

Paso 1: obtener el último snapshot de resultados

base = pilaResultados.back() -> $O(1)$

Paso 2: ordenar resultados por puntaje desc / id asc

sort(base.begin(), base.end(), ...) -> $O(k \log k)$

Paso 3: recortar a 10 elementos

resize(10) -> $O(1)$

Complejidad total: $O(k \log k)$ (dominante: sort)

#

Operación compuesta 3 (extra útil): undo()

Paso 1: pop de pilaUndo -> $O(1)$

Paso 2: borrar del unordered_map -> O(1)
 amortizado
 # Paso 3: borrar del BST por (puntaje,id) -> O(log n)
 promedio / peor O(n)
 # Complejidad total: O(log n) promedio (dominante: borrar en
 BST)

Fase 2: APLICA

APLICA-3

Clase SistemaMX — Inicialización
 # Estructuras inicializadas:
 # 1) catalogo: unordered_map<string, Pelicula> para lookup
 O(1)
 # 2) bst_raiz: puntero a BST (NodoBST) para consultas por
 rango
 # 3) pilaResultados: pila (vector) para snapshots de consultas
 rango
 # 4) pilaUndo: pila (vector) para deshacer inserciones
 #
 # Verificación observada en ejecución:
 # catalogo size: 0
 # bst vacio: 1
 # pilaResultados vacia: 0
 # pilaUndo vacia: 0

APLICA-4

Método agregar_contenido(id,titulo,puntaje) — Inserción
 consistente
 # Valida:
 # - ID duplicado -> invalid_argument
 # - puntaje fuera de [0,10] -> out_of_range
 # Actualiza estructuras (invariante de integración):
 # - catalogo.insert({id, p}) -> Hash (O(1) amort.)
 # - bst_raiz = bst_insertar(bst_raiz, p) -> BST (O(log n)
 prom.)
 # - pilaUndo.push_back({id}) -> Pila undo (O(1))
 # Resultado esperado: después de cada inserción, el contenido es
 visible
 # desde lookup (catalogo) y desde rango (BST), y es reversible
 con undo.

APLICA-5

Método procesar_solicitud(tipo,...) — Enrutador / Fachada

Tipos implementados:

- "lookup" : busca por ID en unordered_map y devuelve vector con 0 o 1 elemento

- "rango" : valida $\text{minv} \leq \text{maxv}$, consulta BST, guarda snapshot en pilaResultados y devuelve resultados

- "top10" : ordena el último snapshot (requiere haber ejecutado "rango" antes) y recorta a 10

- "undo" : revierte la última inserción (borra en hash + borra en BST)

Tipo desconocido: lanza invalid_argument("Tipo desconocido")

Fase 3: REFLEXIONA

```
# REFLEXIONA-1
# Resultados reales de benchmark (microsegundos, us)
#
# | Operación | n=1,000 Mediana(us) | n=1,000 P95(us) | n=10,000 Mediana(us) |
# |-----|-----|-----|-----|
# | lookup   | 0.3645              | 0.5640           | 0.3610                |
# | rango    | 76.0165             | 104.1030         | 808.8930              |
# | top10    | 40.6105             | 59.4830          | 464.8990              |
#
# | n=10,000 P95(us) | Complejidad |
# |-----|-----|
# | 0.4050            | O(1) amort. |
# | 1082.2250         | O(log n + k) prom. (peor O(n+k)) |
# | 635.3440          | O(k log k) (ordena el snapshot del rango) |
```

Observación P95 vs Mediana:

- lookup: P95 cercano a la mediana (bajo jitter).

- rango/top10: P95 mayor por variación del SO/cache y por procesar más datos (k) y ordenar.

#

Cuello de botella (n=10,000):

- rango (Mediana 808.8930 us) es el más costoso -> domina el tiempo del sistema.

Fase 4: VALIDA

VALIDA (V-8)

- # Reporte de 5 casos: 2 dados + 3 edge cases del equipo
- #
- # Caso dado 1: lookup inexistente
- # - Entrada: ID no existente
- # - Esperado: resultado vacío (vector size 0)
- # - Real: OK (si no está en catalogo, no se agrega al vector)
- # - Estado:  PASA
- #
- # Caso dado 2: inserción consistente
- # - Acción: agregar_contenido("VTEST_99", ..., 5.5) y consultar por lookup y por rango
- # - Esperado: visible por lookup y aparece en rango [5.4,5.6]
- # - Real: OK (entra a unordered_map y BST)
- # - Estado:  PASA
- #
- # Edge case 1: rango inválido (minv > maxv)
- # - Entrada: procesar_solicitud("rango","", 9.0, 7.0)
- # - Esperado: lanzar invalid_argument("Rango invalido")
- # - Real: lanza invalid_argument
- # - Estado:  PASA
- #
- # Edge case 2: top10 sin ejecutar rango antes
- # - Entrada: procesar_solicitud("top10") con pilaResultados vacía
- # - Esperado: lanzar runtime_error("Ejecuta rango primero")
- # - Real: lanza runtime_error
- # - Estado:  PASA
- #
- # Edge case 3: undo sin nada que deshacer
- # - Entrada: procesar_solicitud("undo") con pilaUndo vacía
- # - Esperado: lanzar runtime_error("Nada que deshacer")
- # - Real: lanza runtime_error

- Estado:  PASA

VALIDA (V-9)

Invariante cruzada propuesta para este C++:

Inv-1: `catalogo.size() == cantidad_total_de_peliculas_en_BST`

Inv-2: `pilaUndo.size() == catalogo.size()`

Inv-3: BST en orden ascendente por clave (inorder)

#

Resultado esperado tras 50 inserciones aleatorias:

- Inv-1: 

- Inv-2: 

- Inv-3: 

#

Nota: Para Inv-1 en este diseño, cada `NodoBST` guarda `vector<Pelicula>`,

así que contar "nodos" no basta; se debe contar total de películas en todos los vectores.

`#include <iostream>`

`#include <unordered_map>`

`#include <vector>`

`#include <string>`

`#include <algorithm>`

`#include <stdexcept>`

`#include <chrono>`

`#include <functional>`

`#include <random>`

`#include <iomanip>`

`#include <sstream>`

`using namespace std;`

`using namespace std::chrono;`

```

// =====
// StreamMX (Modelo simplificado)
// =====

struct Pelicula {
    string id;
    string titulo;
    double puntaje;
};

// Nodo BST
struct NodoBST {
    double clave;
    vector<Pelicula> peliculas;
    NodoBST* izq;
    NodoBST* der;

    NodoBST(double c, const Pelicula& p)
        : clave(c), peliculas{p}, izq(nullptr), der(nullptr) {}
};

struct OperacionUndo {
    string idInsertado;
};

class SistemaMX {
public:
    unordered_map<string, Pelicula> catalogo;
    NodoBST* bst_raiz;
    vector<vector<Pelicula>> pilaResultados;
    vector<OperacionUndo> pilaUndo;
};

```



```
SistemaMX() : bst_raiz(nullptr) {}
```

```
~SistemaMX() {  
    liberarBST(bst_raiz);  
}
```

```
// =====
```

```
// Inserción consistente
```

```
// =====
```

```
bool agregar_contenido(const string& id,  
                        const string& titulo,  
                        double puntaje) {
```

```
    if (catalogo.find(id) != catalogo.end()) {  
        throw invalid_argument("ID duplicado: " + id);  
    }
```

```
    if (puntaje < 0.0 || puntaje > 10.0) {  
        throw out_of_range("Puntaje fuera de rango");  
    }
```

```
Pelicula p{id, titulo, puntaje};
```

```
// Hash  
catalogo.insert({id, p});
```

```
// BST  
bst_raiz = bst_insertar(bst_raiz, p);
```

```
// Undo  
pilaUndo.push_back({id});
```

```
return true;
```

```
}
```

```
// =====
```

```
// Router / fachada
```

```
// =====
```

```
vector<Pelicula> procesar_solicitud(const string& tipo,  
                                   const string& paramStr = "",  
                                   double minv = 0.0,  
                                   double maxv = 0.0) {
```

```
    // LOOKUP
```

```
    if (tipo == "lookup") {  
        vector<Pelicula> out;
```

```
        auto it = catalogo.find(paramStr);
```

```
        if (it != catalogo.end()) {  
            out.push_back(it->second);  
        }
```

```
        return out;  
    }
```

```
    // RANGO
```

```
    if (tipo == "rango") {  
  
        if (minv > maxv) {  
            throw invalid_argument("Rango invalido");  
        }
```

```
        vector<Pelicula> resultados;
```

```
        bst_rango(bst_raiz, minv, maxv, resultados);
```

```

        pilaResultados.push_back(resultados);

        return resultados;
    }

// TOP10
if (tipo == "top10") {

    if (pilaResultados.empty()) {
        throw runtime_error("Ejecuta rango primero");
    }

    vector<Pelicula> base = pilaResultados.back();

    sort(base.begin(), base.end(),
        [](const Pelicula& a, const Pelicula& b) {

            if (a.puntaje != b.puntaje) {
                return a.puntaje > b.puntaje;
            }

            return a.id < b.id;
        });

    if (base.size() > 10) {
        base.resize(10);
    }

    return base;
}

// UNDO

```

```

    if (tipo == "undo") {
        deshacer_ultima_insercion();
        return {};
    }

    throw invalid_argument("Tipo desconocido");
}

private:

// =====
// BST insertar
// =====
NodoBST* bst_insertar(NodoBST* nodo, const Pelicula& p) {

    if (!nodo) {
        return new NodoBST(p.puntaje, p);
    }

    if (p.puntaje < nodo->clave) {
        nodo->izq = bst_insertar(nodo->izq, p);
    }
    else if (p.puntaje > nodo->clave) {
        nodo->der = bst_insertar(nodo->der, p);
    }
    else {
        nodo->peliculas.push_back(p);
    }

    return nodo;
}

// =====

```

```

// BST rango
// =====
void bst_rango(NodoBST* nodo,
               double minv,
               double maxv,
               vector<Pelicula>& out) {

    if (!nodo) return;

    if (minv < nodo->clave) {
        bst_rango(nodo->izq, minv, maxv, out);
    }

    if (minv <= nodo->clave && nodo->clave <= maxv) {
        out.insert(out.end(),
                   nodo->peliculas.begin(),
                   nodo->peliculas.end());
    }

    if (nodo->clave < maxv) {
        bst_rango(nodo->der, minv, maxv, out);
    }
}

// =====
// Liberar BST
// =====
void liberarBST(NodoBST* nodo) {

    if (!nodo) return;

    liberarBST(nodo->izq);
    liberarBST(nodo->der);
}

```

```

    delete nodo;
}

// =====
// Undo
// =====
void deshacer_ultima_insercion() {

    if (pilaUndo.empty()) {
        throw runtime_error("Nada que deshacer");
    }

    string id = pilaUndo.back().idInsertado;
    pilaUndo.pop_back();

    auto it = catalogo.find(id);

    if (it == catalogo.end()) {
        throw runtime_error("Inconsistencia");
    }

    Pelicula p = it->second;

    catalogo.erase(it);

    bst_raiz = bst_borrar(bst_raiz, p.puntaje, id);
}

// =====
// BST borrar
// =====
NodoBST* bst_borrar(NodoBST* nodo,

```

```

        double clave,
        const string& id) {

    if (!nodo) return nullptr;

    if (clave < nodo->clave) {
        nodo->izq = bst_borrar(nodo->izq, clave, id);
        return nodo;
    }

    if (clave > nodo->clave) {
        nodo->der = bst_borrar(nodo->der, clave, id);
        return nodo;
    }

    auto& v = nodo->peliculas;

    v.erase(remove_if(v.begin(), v.end(),
        [&](const Pelicula& p) {
            return p.id == id;
        })), v.end());

    if (!v.empty()) {
        return nodo;
    }

    // Caso hoja
    if (!nodo->izq && !nodo->der) {
        delete nodo;
        return nullptr;
    }

    // Un hijo

```

```
if (!nodo->izq) {  
    NodoBST* tmp = nodo->der;  
    delete nodo;  
    return tmp;  
}
```

```
if (!nodo->der) {  
    NodoBST* tmp = nodo->izq;  
    delete nodo;  
    return tmp;  
}
```

```
// Dos hijos  
NodoBST* succParent = nodo;  
NodoBST* succ = nodo->der;
```

```
while (succ->izq) {  
    succParent = succ;  
    succ = succ->izq;  
}
```

```
nodo->clave = succ->clave;  
nodo->peliculas = succ->peliculas;
```

```
if (succParent->izq == succ) {  
    succParent->izq =  
        bst_borrar(succ, succ->clave, "");  
}  
else {  
    succParent->der =  
        bst_borrar(succ, succ->clave, "");  
}
```



```

        return nodo;
    }
};

//
=====

=====
// Benchmark
//
=====

=====

struct BenchResult {
    double median_us;
    double p95_us;
    double min_us;
};

BenchResult medir_p95(function<void()> fn,
                    int warmup = 30,
                    int reps = 300) {

    for (int i = 0; i < warmup; i++) {
        fn();
    }

    vector<double> muestras;
    muestras.reserve(reps);

    for (int i = 0; i < reps; i++) {

        auto t0 = steady_clock::now();

```

```

    fn();

    auto dt =
        duration_cast<nanoseconds>(
            steady_clock::now() - t0
        ).count() / 1000.0;

    muestras.push_back(dt);
}

sort(muestras.begin(), muestras.end());

double median;

if (reps % 2 == 0) {
    median =
        (muestras[reps/2 - 1] + muestras[reps/2]) / 2.0;
}
else {
    median = muestras[reps/2];
}

int p95_idx = (int)(0.95 * reps);

if (p95_idx >= reps) {
    p95_idx = reps - 1;
}

return {
    median,
    muestras[p95_idx],
    muestras[0]
};

```

```
}
```

```
struct BenchmarkRow {  
    string op;  
    int n;  
    BenchResult r;  
};
```

```
static string fmt_us(double us) {  
  
    ostringstream oss;  
  
    oss << fixed << setprecision(4) << us;  
  
    return oss.str();  
}
```

```
//  
=====
```

=====

```
// Benchmark del sistema  
//  
=====
```

=====

```
vector<BenchmarkRow> benchmark_sistema(int n,  
                                       SistemaMX& sistema) {
```

```
    mt19937 rng(12345);
```

```
    uniform_real_distribution<double>  
        distPts(1.0, 10.0);
```

```
vector<string> ids;
ids.reserve(n);

// Poblar
for (int i = 0; i < n; i++) {

    string id = "C" + to_string(i);

    double pts = distPts(rng);

    sistema.agregar_contenido(
        id,
        "Titulo_" + to_string(i),
        pts
    );

    ids.push_back(id);
}

uniform_int_distribution<int>
    distIdx(0, n - 1);

string id_muestra = ids[distIdx(rng)];

// Activar snapshot para top10
(void) sistema.procesar_solicitud(
    "rango",
    "" ,
    7.0,
    9.0
);

vector<BenchmarkRow> rows;
```

```
// LOOKUP
rows.push_back({
    "lookup",
    n,
    medir_p95([&]() {
        (void) sistema.procesar_solicitud(
            "lookup",
            id_muestra
        );
    })
});
```

```
// RANGO
rows.push_back({
    "rango",
    n,
    medir_p95([&]() {
        (void) sistema.procesar_solicitud(
            "rango",
            "",
            7.0,
            9.0
        );
    })
});
```

```
// TOP10
rows.push_back({
    "top10",
    n,
    medir_p95([&]() {
        (void) sistema.procesar_solicitud(
```

```

        "top10"
    );
})
});

return rows;
}

//
=====

=====
// Imprimir tabla
//
=====

=====

static void imprimir_tabla(
    const vector<BenchmarkRow>& rows) {

    cout << left << setw(12) << "Operacion"
        << right << setw(10) << "n"
        << right << setw(16) << "Mediana(us)"
        << right << setw(14) << "P95(us)"
        << right << setw(14) << "Min(us)"
        << "\n";

    cout << string(66, '-') << "\n";

    for (const auto& row : rows) {

        cout << left << setw(12) << row.op
            << right << setw(10) << row.n
            << right << setw(16)

```

```

        << fmt_us(row.r.median_us)
        << right << setw(14)
        << fmt_us(row.r.p95_us)
        << right << setw(14)
        << fmt_us(row.r.min_us)
        << "\n";
    }
}

//
=====
=====
// MAIN UNIFICADO
//
=====
=====

int main() {

                                cout          <<
"===== \n";
    cout << " DEMO BASICA SISTEMAMX \n";

                                cout          <<
"===== \n";

    SistemaMX s;

    // Inicializaci3n
    cout << "catologo size: "
        << s.catologo.size() << "\n";

    cout << "bst vacio: "
        << (s.bst_raiz == nullptr) << "\n";

```

```
cout << "pilaResultados vacia: "  
    << s.pilaResultados.size() << "\n";
```

```
cout << "pilaUndo vacia: "  
    << s.pilaUndo.size() << "\n";
```

```
// Inserciones
```

```
s.agregar_contenido("MX001", "El Laberinto", 8.5);  
s.agregar_contenido("MX002", "Noche Polar", 7.2);  
s.agregar_contenido("MX003", "Codigo Cero", 9.1);  
s.agregar_contenido("MX004", "Delta", 6.0);
```

```
// Lookup
```

```
auto r1 = s.procesar_solicitud(  
    "lookup",  
    "MX001"  
);
```

```
cout << "\nlookup MX001 -> "  
    << (r1.empty() ? "NO" : r1[0].titulo)  
    << "\n";
```

```
// Rango
```

```
auto r2 = s.procesar_solicitud(  
    "rango",  
    "",  
    7.0,  
    9.0  
);
```

```
cout << "rango [7,9] -> "  
    << r2.size()
```



```
<< " resultados\n";
```

// Top10

```
auto top = s.procesar_solicitud("top10");
```

```
cout << "top10 -> "
```

<< top.size()

```
<< "\n";
```

// Undo

```
s.procesar_solicitud("undo");
```

```
auto r3 = s.procesar_solicitud(
```

"lookup",

"MX004"

) ;

```
cout << "despues de undo -> "
```

```
<< (r3.empty() ? "NO EXISTE" : "EXISTE")
```

```
<< "\n";
```

```
cout << "\nDemo basica OK\n";
```

/// =====

// BENCHMARKS

/// =====

cout <<

```
"\n=====\\n";
```

```
cout << " BENCHMARK n = 1,000\n";
```

cout <<

```
"===== \n";
```

```

{
    SistemaMX s1;

    auto r1 = benchmark_sistema(1000, s1);

    imprimir_tabla(r1);
}

cout <<
"\n===== \n";
cout << " BENCHMARK n = 10,000 \n";
cout <<
"===== \n";

{
    SistemaMX s2;

    auto r2 = benchmark_sistema(10000, s2);

    imprimir_tabla(r2);
}

return 0;
}

=====
DEMO BASICA SISTEMAMX
=====
catalogo size: 0
bst vacio: 1
pilaResultados vacia: 0
pilaUndo vacia: 0

lookup MX001 -> El Laberinto

```

rango [7,9] -> 2 resultados
top10 -> 2
despues de undo -> NO EXISTE

Demo basica OK

=====

BENCHMARK n = 1,000

=====

Operacion	n	Mediana(us)	P95(us)	Min(us)

lookup	1000	0.3280	0.3840	0.3100
rango	1000	76.5130	109.6390	74.2530
top10	1000	40.8890	59.4020	39.1660

=====

BENCHMARK n = 10,000

=====

Operacion	n	Mediana(us)	P95(us)	Min(us)

lookup	10000	0.3640	0.4070	0.3110
rango	10000	868.9415	918.5160	814.4350
top10	10000	477.6630	513.3130	448.4950